

Folang Programming Language

Folang is a general-purpose programming language designed to be **expressive, consistent, and extensible**, merging functional fluency with object-centric abstractions.

Design Overview

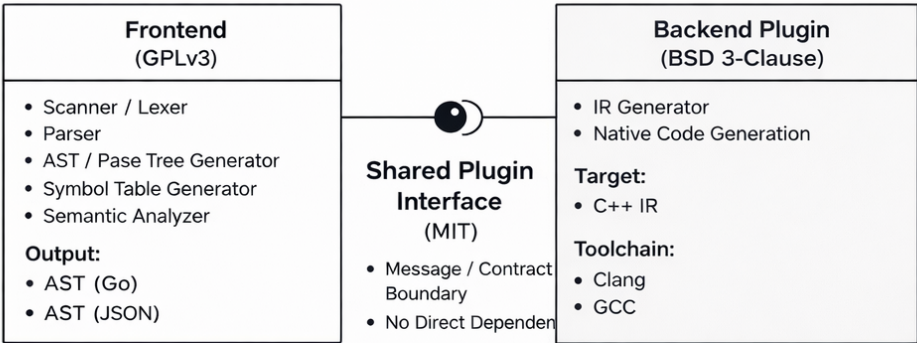


Figure: Frontend and Backend are independent components connected exclusively through a shared plugin interface, which acts as a strict integration and licensing boundary.

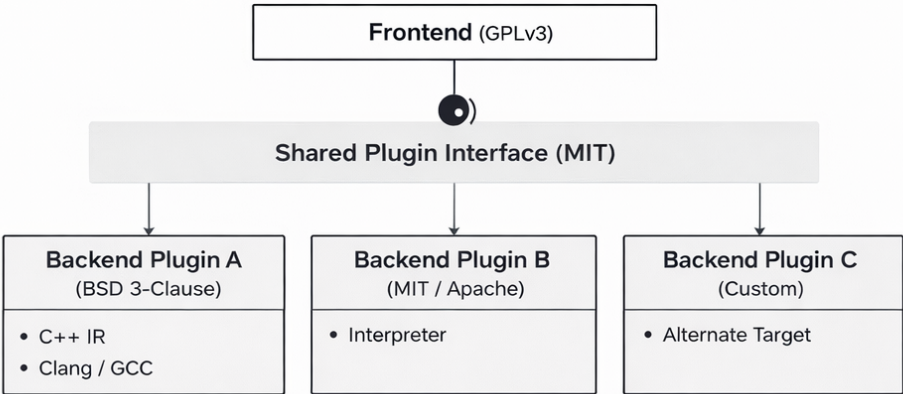


Figure: The Backend itself is a plugin. Multiple backend implementations can coexist or be swapped without modifying the Frontend.

FoLang follows a deliberately different approach from conventional programming language designs. The system is structured to ensure **clear separation of concerns, license isolation, and extensibility through well-defined integration boundaries**.

1. Frontend

The Frontend is responsible for source-level analysis and semantic processing of FoLang programs.

Components

- Scanner / Lexer
- Parser
- AST / Parse Tree Generator
- Symbol Table Generator
- Semantic Analyzer

Implementation

- Implemented in **Go**
- Generates AST representations in **Go structures** or **plain JSON**

License

- **GNU General Public License v3 (GPLv3)**

Why the Frontend Is Not Pluggable

FoLang's syntax is **fixed and closed**. All language extensibility — operators, generics, macros, annotations, type extensions, custom matchers — is expressed through the built-in `@co.dap` and `@co.ddap` annotation system within the fixed syntax. There is no mechanism to introduce new syntax from outside the language.

Because no external component can alter what the frontend parses or how it builds the AST, a frontend plugin architecture would add complexity with no benefit. The frontend is therefore a **single, fixed, non-pluggable component**.

Extensibility need	How FoLang handles it
New syntax	Not allowed — syntax is fixed
New operators	<code>@co.dap.operator</code> — declared in-language
Generics	<code>@co.dap.generic</code> — annotation-driven
Macros	<code>@co.dap.macro</code> — AST manipulation in-language
Annotations / decorators	<code>@co.dap.annotation</code> etc. — in-language
Type extensions	<code>@co.dap.extension</code> — in-language
Custom matchers	<code>@co.dap.matcher</code> — in-language

2. Backend

The Backend is responsible for transforming validated frontend output into executable artifacts.

The Backend itself is implemented and integrated as a **plugin**, using the same shared plugin interfaces available to third parties. The out-of-the-box (OOTB) backend is provided as a **default plugin implementation**.

Components

- Intermediate Representation (IR) Generator
- Native Binary Executable Generation

Implementation

- Backend orchestration and plugin integration implemented in **Go**

- Code generation target is **C++**
- Uses **Clang** or **GCC** to generate native binaries from generated C++ IR

License

- **BSD 3-Clause License**
-

Why the Backend Is a Plugin

As illustrated in the architecture diagrams, the Backend is intentionally treated as a **pluggable component** rather than a privileged or tightly coupled part of the system.

This design ensures that the Frontend depends only on **stable interfaces**, not on any specific backend implementation. As a result, backend implementations can be added, replaced, or evolved independently without requiring changes to the Frontend.

Treating the Backend as a plugin also establishes a clear **integration and licensing boundary**, enabling multiple backend implementations — each with different execution models, targets, or licenses — to coexist behind the same shared interface.

3. Shared Interfaces

The Shared layer defines stable **contracts and interfaces** that any backend plugin must conform to. It is not itself a plugin — it is the integration boundary between the Frontend and any Backend implementation.

Purpose

- Defines the HIR schema that the Frontend produces and any Backend must consume
- Defines the IPC protocol and wire format contract
- Enables third parties to build custom backend implementations against a stable interface
- Acts as the strict boundary that allows Frontend and Backend to evolve independently

What Third Parties Can Do

Using the shared interfaces, third parties can provide custom Backend implementations in any language — as long as they conform to the HIR schema and IPC protocol declared here.

License

- **MIT License**
-

4. Plugin Configuration

FoLang uses a **minimal and explicit configuration file** to define how the Frontend and Backend are connected at runtime.

Each FoLang binary distribution includes:

- **Exactly one Frontend** — fixed, not configurable
- **Exactly one Backend** — selected via configuration

There is **no runtime plugin selection or discovery**. Different backend implementations are achieved by distributing different FoLang binaries or swapping the backend plugin.

Backend Kinds

There are two kinds of backend:

Kind 1 — Plugin backend implemented in the same language as the frontend (Go). Integrates directly via shared interfaces which are already versioned for backward compatibility. The frontend does not emit protobuf — it communicates through the shared

interfaces directly. Config needs only plugin path and version.

Kind 2 — Independent backend implemented in any language. The frontend emits HIR over an IPC boundary in the declared wire format. Config declares the full protocol, schema, and wire format.

Configuration File Structure

Kind 1 — Plugin backend

```
{
  "plugin": "libs/foolang-plugin",
  "version": "v1"
}
```

Kind 2 — Independent backend

```
{
  "protocol": "foolang-plugin/1.0",
  "hir_schema": "foolang-hir/1",
  "wire": "protobuf"
}
```

Licensing Summary

Layer	Pluggable?	Responsibility	Implementation	License
Frontend	✗ Fixed	Parsing and semantic analysis	Go	GPLv3
Backend	✓ Plugin	IR processing and native code generation	Go (orchestration) + C++ (target)	BSD 3-Clause
Shared	✓ Interfaces	Backend integration contracts and HIR schema	Go	MIT

6. Capability Security Model

FoLang's compiler ships with all language features compiled in but **systems and FFI features are disabled by default**. The compiler has no hardcoded keys — capability configuration happens entirely at install time. This moves authorization from source code (developer-controlled) to the compiler installation (organization-controlled).

Feature Tiers

Tier	Features	Default State
application	All standard language features, <code>co.net</code> , <code>co.core</code> , <code>co.encoding</code> , <code>co.crypto</code> , etc.	✓ Always enabled
systems	Raw pointers, pointer arithmetic, <code>co.sys.unsafe</code> , MMIO, heap allocators	🚫 Disabled — requires install-time configuration
ffi	<code>@co.dap.native</code> , <code>co.sys.ffi</code> , extern types, <code>co.lang.void</code> pointers, C ABI	🚫 Disabled — requires install-time configuration

Quick Start

Hello World

```
// hello.fol – entry file, no annotation needed
co.out.println("Hello FoLang!")
```

Or with an alias for shorter form:

```
@co.ddap.alias(co.out, as="out")

out.println("Hello FoLang!")
```

Variables

```
// typed
name co.lang.string = "Rao";
age  co.lang.int    = 30;

// inferred from value – := errors if already declared
name := "Rao";
age  := 30;

// assign if not defined, otherwise reassign
name ?= "Kumar";
```

FoLang Philosophy — Uniform Object Model

Core Principle

Everything in **FoLang** is an object. That is the reason for the name **FO — Functional Objects**.

FoLang gives the programmer one uniform object model across all types instead of forcing them to switch between separate type families for:

- ordinary values
- immutable values
- shared/concurrent values
- copy-on-write values
- literal values

The programmer writes against a single conceptual model and opts into the required object behaviour only when needed.

Uniform Object Principle

In FoLang, **everything is an object**.

This includes:

- scalar objects
- collection objects
- user-defined objects
- ADT objects

- function objects

Functions are objects in the same sense as all other values.

This is true for both **named functions** and **anonymous functions**.

FoLang does not introduce a separate semantic model for functions, scalars, UDTs, or ADTs.

They all follow the same core object principles:

- assignment copies references
- `==` compares values
- mutation is applied through the object
- behaviour policies such as immutability, shared, copy-on-write, and literal conversion apply uniformly where meaningful

So in FoLang, the programmer does not need one mental model for data objects and another for function values.

The language treats them under one consistent object model.

1. Default Object Model

Every value in FoLang is an object and is **mutable by default**.

This includes:

```
co.lang.int, co.lang.float, co.lang.string → built-in scalar types
user-defined types (structs, classes, ADTs) → user-defined objects
co.core.list, co.core.map, co.core.array → built-in collections
```

Example:

```
positive_int co.lang.int = 10;
```

`positive_int` denotes an object.

Its current value is `10`.

By default, it is mutable.

2. Assignment, Aliasing, and Mutation

FoLang distinguishes three related ideas:

- **assignment**
- **aliasing**
- **mutation**

2.1 Assignment copies references

When one object variable is assigned to another, FoLang copies the **reference**, not the internal contents.

```
b := a
```

After this, `a` and `b` refer to the same object.

2.2 Mutation changes object contents

If two names refer to the same object, mutating through one name is visible through the other.

```

Employee co.lang.class = {
    Name co.lang.string
}

a Employee = { Name = "Kamesh" };
b := a;
c Employee = { Name = "Kamesh" };

a == b;    // true
a == c;    // true
b == c;    // true

b.Name = "Ramesh";    // changes a.Name too
c.Name = "Ramesh";    // does not change a.Name

```

Why:

- **a** and **b** are the **same object**
- **c** is a **different object** with the same earlier value

2.3 == means value equality

In FoLang, == compares **values**, not object identity.

That rule is uniform across all object kinds, including built-in types and user-defined types.

So:

```
a == b
```

means:

- compare contents deeply
- return **true** when the values match
- even if **a** and **b** are not the same object

2.4 Mutation accessors

The accessor used for mutation depends on the object type:

```

scalar (int, float, bool, string, char, byte)  → .value
struct field                                   → .fieldname
class member                                   → .membername
map entry                                       → .key
array / list element                           → .index

```

Examples:

```

count.value = 30
emp.name = "Rao"
marks.math = 95
nums.0 = 42

```

2.5 Function-call behaviour

Function parameters introduce a **new local binding**.

So:

- rebinding a parameter does **not** affect the caller
- mutating the object through the parameter **does** affect the caller if both still refer to the same object

```
checkPositive(a co.lang.int)->(co.lang.bool) = {  
  a = 20;           // local rebinding only – caller unchanged  
  a.value = 30;     // object mutation – caller sees 30  
}
```

If `positive_int` is passed to `checkPositive(a)`:

```
a = 20           → local rebinding only  
a.value = 30     → mutates the passed object
```

3. Literal Objects

All literals in FoLang create **Literal Objects**.

```
"Kumar"  → Literal Object – string  
42       → Literal Object – int  
true     → Literal Object – bool
```

A Literal Object is an **anonymous object created from a literal expression**.

Literal Objects participate in value equality just like every other object in FoLang:

```
10 == 10  // true
```

In FoLang, `==` compares **values**, not object identity.

So two literal expressions with the same value compare equal, but that does not mean they are the same object.

Binding a Literal Object

When a literal is assigned to a named object, the name refers to the object created from that literal expression.

```
a co.lang.int = 10;
```

Here, `a` refers to the anonymous integer object created from literal `10`.

So:

```
a.value = 30;
```

mutates that same object, and `a` now has value `30`.

The important points are:

- literal expressions create anonymous objects
- literal-created objects are still ordinary objects unless another policy is applied
- `==` compares values, not identity
- once bound to a name, a literal-created object behaves like any other normal mutable object
- immutability applies only when explicitly requested, for example through `makeImmutable(...)`

Why Literal Objects Cannot Be Mutated Directly

Mutation happens in two ways

1. Through Rebinding (only for LValues not RValue)

```
ex:  a co.lang.int =10;
      a =20;
```

Above is Valid, but when we try directly on Literal object 10 it is RValue not LValue, has no handle/named ref of its own (as it is anonymous object) so cannot be rebounded.

2. Through proerty or method

```
ex:  a co.lang.int = 10;
      a.value = 20;
```

Above is valid, but when we try directly on Literal objects they doesn't have Value property.

```
a co.lang.int = 10;
a.value = 30;
```

is valid after binding, a bare literal such as `10` cannot be mutated directly because there is no name or handle through which to perform mutation.

Not the Same as `toSnapshot(...)`

A literal object created by a literal expression is **not** the same thing as the internal snapshot representation produced by `co.utils.toSnapshot(...)`.

- literal expression → anonymous object
- `toSnapshot(x)` → internal snapshot representation used to reconstruct a fresh local object later

These are related concepts, but they are not the same mechanism.

Summary

- literal objects are real objects
- literal expressions create anonymous objects
- identical literals compare equal by value
- identical literals are not automatically the same object
- literal-created objects are mutable by default once bound to a handle
- only `makeImmutable(...)` makes an object immutable

4. Object Behaviour Policies

Any object can be given a behaviour policy using `co.utils.*`.

All four policy calls are **in-place transformations**:

- the object itself changes behaviour kind
- there is no wrapper object
- there is no alternate binding to capture
- the original name now refers to the transformed object

All policies are **deep by default**.

They flow through nested structs, members, collection elements, and all reachable objects in the graph unless the specification later states otherwise.

4.1 Immutable

```
co.utils.makeImmutable(positive_int)
```

After this call the object itself is immutable.

This is an in-place transformation, not a wrapper.

Any attempt to mutate the object or any reachable object beneath it is a **compiler error** where detectable statically, and a **runtime error** otherwise.

```
positvie_int co.lang.int = 10;
positive_int.value = 30    // ✗ compiler error
positive_int = 30         // ✗ compiler error
```

For nested objects:

```
Employee co.lang.struct = { address Address; }
Address co.lang.Address = {city co.lang.string; state co.lang.string; lane
co.lang.string;pin co.lang.string;}

emp Employee = Employee{
    address:Address{
        city: "Pune";
    }
}
co.utils.makeImmutable(emp);

emp = Employee{
    address:Address{
        city: "ABC";
    }
} // ✗ compiler error

emp.address = newAddress           // ✗ compiler error
emp.address.city.value = "Mumbai"  // ✗ compiler error
```

Immutability is deep and total.

4.2 Value Immutable

```
positive_int co.lang.int = 20;

co.utils.makeValueImmutable(positive_int);

positive_int.value = 30; // ✗ current object is immutable
positive_int = 30;      // ✔ binding may reference a new object

co.utils.makeValueImmutable(emp);

emp.address.city = "ABC";      // ✗
emp.address.city.value = "ABC"; // ✗

emp = Employee{
  address: Address{
    city: "ABC";
  }
}; // ✔
```

Difference between Immutable and Immutable Value

```
makeValueImmutable(x)
└─ current object graph cannot change

makeImmutable(x)
├─ current object graph cannot change
└─ binding cannot be reassigned
```

Table

Operation	Binding	Current value/object graph
<code>makeValueImmutable(x)</code>	Mutable	Immutable
<code>makeImmutable(x)</code>	Immutable	Immutable

4.3 Shared

```
co.utils.makeShared(positive_int)
```

A Shared Object is safe for concurrent and multi-threaded use.

The programmer continues to use the same object model — same accessors, same syntax — while FoLang guarantees the required safety properties internally.

Shared behaviour is deep:

- all reachable objects within the shared object are also shared

Note on Analogies

Comparisons to things like Java's `AtomicInteger` or `ConcurrentHashMap` are **analogies only**.

They may help explain the concept, but they are **not part of the formal language contract**.

FoLang specifies the behavioural guarantee, not the exact runtime implementation.

A good explanatory statement is:

A shared `co.lang.int` may be thought of similarly to an atomic integer, and a shared map may be thought of similarly to a concurrent map. These comparisons are explanatory only. FoLang does not require any particular internal runtime representation.

4.4 CopyOnWrite

```
co.utils.copyOnWrite(positive_int)
```

A `CopyOnWrite` Object passes by reference like a normal object.

The copy is deferred.

It is created only when mutation is attempted in a context that must not affect the original source object.

```
process(a co.lang.int)->>() = {
    a.value = 99;    // deep copy is made here – caller's object unchanged
}

co.utils.copyOnWrite(positive_int)
process(positive_int)

// positive_int still holds its old value
```

Copy-on-write is deep.

When a copy is made, the entire reachable object graph is copied.

Cyclic references must be handled by the runtime/compiler's structural clone logic with proper identity tracking.

4.5 toSnapshot

```
co.utils.toSnapshot(positive_int)
```

`toSnapshot` converts an object into a **snapshot representation** — a value descriptor, not a live object. The snapshot representation itself cannot be mutated.

When passed to a function, the compiler/runtime uses the snapshot representation to construct a fresh independent local variable bound to the parameter name. That local is a normal mutable object with no shared identity with the original. All of this happens automatically — the developer writes only `co.utils.toSnapshot(positive_int)`.

```
process(a co.lang.int)->>() = {
    a.value = 99;    // mutates the fresh local – positive_int completely unaffected
}

process(co.utils.toSnapshot(positive_int))

// positive_int unchanged
```

The flow:

```
positive_int
  ↓ co.utils.toSnapshot(positive_int)
  snapshot representation      ← value descriptor – immutable by nature, not by
policy
  ↓ passed to function
  compiler constructs fresh local 'a' from the snapshot representation
  'a' is a new independent normal object – mutations never reach positive_int
```

`toSnapshot` is deep — the snapshot representation covers the entire reachable object graph.

5. Policy Summary

Object Kind	Mutation allowed	Caller sees mutation	Thread safe	Copy on write	Deep
Literal expression object	× directly in source — no handle	—	value-like use	—	✓
Normal (default)	✓ via accessor	✓	×	×	—
Immutable	×	—	✓	—	✓
Shared	✓ via accessor	✓	✓	×	✓
CopyOnWrite	✓ on own copy	×	✓	✓	✓
toSnapshot result	✓ on reconstructed local object	×	independent snapshot	—	✓

6. No Type Fragmentation

FoLang deliberately avoids special types for mutability or concurrency concerns.

There is no need for separate public type families such as:

```
ImmutableInt
SharedMap
CopyOnWriteList
AtomicInt
ConcurrentMap
```

Instead, FoLang keeps the type model uniform and applies behaviour policies through `co.utils.*`.

In many ecosystems, a programmer must constantly choose between:

```
normal integer      vs atomic integer
normal map          vs concurrent map
mutable value       vs immutable wrapper
raw structure       vs copy-on-write structure
```

FoLang instead aims to provide:

- one object model

- one programming style
- one mental model

while still allowing the programmer to opt into immutability, sharing, copy-on-write, or literal conversion when needed.

7. What Still Needs Precision

The philosophy is sound, but the formal specification still needs to define these precisely:

aliasing behaviour	→ when two names refer to the same object
rebinding vs mutation interaction	→ edge-case coverage
deep policy propagation rules	→ exactly which objects are reachable
cyclic reference handling in COW	→ identity-tracking specification
visibility of mutation across calls	→ formal function-call model
identity vs value equality	→ what operators or built-ins expose identity
policy stacking	→ can an object be both shared and COW? can an object be both shared and immutable?

8. Formal Philosophy Statement

In FoLang, everything is an object and objects are mutable by default.

Assignment copies references, while `==` compares values deeply.

Developers may opt into immutability, shared behaviour, copy-on-write behaviour, or literal conversion depending on their needs.

All behaviour policies are deep — they apply to the entire reachable object graph unless stated otherwise by the formal specification.

This model is uniform across all types, so programmers do not need separate mental models or separate type families for ordinary, immutable, concurrent, or snapshot-oriented use.

Familiar analogies such as atomic integers or concurrent maps may help explain the design, but they are not part of the formal implementation contract.

FoLang Packages, Imports, and Libraries

Packages

Package Identity

A subfolder containing `.fol` files **is** a package.

- Dot paths start from subfolders.
- The project root is **not** a package.
- The root folder name never appears in any package dot path.

Examples:

```
/appl/hr/           -> package "hr"
/appl/hr/employee/ -> package "hr.employee"
/appl/auth/         -> package "auth"
```

The project root itself is **not** a package.

It may contain:

- the application entry file
- the packaged library surface file when the project itself is a library
- subfolders that define packages or same-owner source libraries

Source Files

A source `.fol` file contains declarations only.

Allowed at top level:

- structs
- classes
- modules
- interfaces
- signatures
- enums / ADTs
- type aliases / newtypes
- functions
- annotations attached to declarations

Forbidden at top level:

- package declarations
- executable statements
- project metadata
- library metadata except in a library surface file

Example:

```
// hr/employee/Employee.fol
Employee co.lang.struct = {
  id    co.lang.int;
  name  co.lang.string;
}
```

```
// hr/employee/EmpService.fol
getEmployee(id co.lang.int)->(Employee) = {
  this.return Employee{ id: 1, name: "Rao" };
}
```

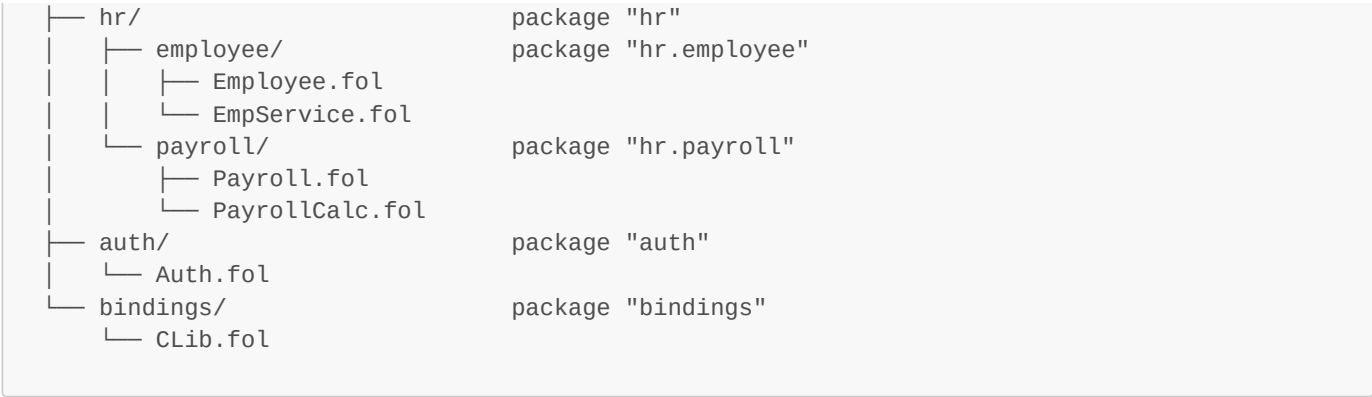
Multi-File Packages

Multiple `.fol` files in the same subfolder automatically belong to the same package:

```
hr/employee/
├─ Employee.fol      → hr.employee
├─ EmpService.fol    → hr.employee
└─ EmpValidator.fol  → hr.employee
```

Application Project Layout

```
/appl/
├─ app.fol           ← entry file – not a package
```



Library Project Layout



Consumer only sees what `hrlib.fol` declares. All subfolders are internal.

Package Access Rules

Four access levels control visibility across package boundaries:

Annotation	Same Package	Parent	Sub-Package	Unrelated	Entry / Surface
@co.dap.public	✓	✓	✓	✓	✓
@co.dap.package	✓	✓	✓	×	×
@co.dap.protected	✓	×	✓	×	×
@co.dap.private	✓	×	×	×	×

Meaning:

- `@co.dap.public` -> visible everywhere
- `@co.dap.package` -> visible across the package family
- `@co.dap.protected` -> visible downward into subpackages only
- `@co.dap.private` -> visible only inside the declaring package

Example:

```
// hr/Employee.fol - package "hr"

@co.dap.public
getEmployee(id co.lang.int)->(Employee) = { ... }    // anyone can call

@co.dap.package
validateId(id co.lang.int)->(co.lang.bool) = { ... } // hr, hr.employee, hr.payroll

@co.dap.protected
```

```
baseQuery()->(co.lang.string) = { ... }           // hr.employee and sub-packages only
                                                    // hr cannot call sub-package
                                                    // package.private declarations

@co.dap.private
cacheMap co.lang.auto = ...                       // hr only
```

co.* Paths and Aliases

co.* Is Always Available

All `co.*` paths are part of the language and are always in scope. They are never imported through `@co.ddap.import`.

```
co.out.println("hello")
co.in.readln()
x co.lang.int = 42;
```

Being **in scope** does not automatically mean **permitted in every context**.

A package rule, library kind, or other compiler restriction may still forbid use of a particular `co.*` facility. In such cases the name is still known to the compiler, but its use is rejected at compile time.

@co.ddap.alias

Use aliases only to shorten `co.*` paths.

```
@co.ddap.alias(co.out, as="out")
@co.ddap.alias(co.core.list, as="list")
@co.ddap.alias(co.encoding, as="enc")

out.println("hello")
list.of(1, 2, 3)
enc.json.serialize(data)

// full form still works alongside
co.out.println("world")
```

Rules:

- target must be a `co.*` path
- alias must be a valid identifier
- alias scope is file-local
- aliasing user packages is not allowed; use `as=` in `@co.ddap.import`
- duplicate aliases in the same file are compiler errors

Imports

FoLang supports three import forms.

1. Normal Package Import

Use this for ordinary project packages.

```
@co.ddap.import(package="hr.employee", as="emp")

e := emp.getEmployee(1);
co.out.println(e.name);
```

Resolution:

```
package="hr.employee" -> /appl/hr/employee/
```

2. Source Library Import

Use this for same-owner workspace libraries whose source is available.

```
@co.ddap.import(package="com.abc.ffi", library=true, expect="ffi", as="ffilib")
```

Resolution:

```
package="com.abc.ffi", library=true -> /appl/com/abc/ffi.fol
```

The resolved file must be a library surface file:

```
@co.dap.library(type="ffi")
```

Meaning:

- **package** gives the logical library path
- **library=true** means the leaf resolves to a single **.fol** surface file, not a folder
- **expect** is an import-site assertion; the compiler checks it against the actual library type
- only the surface file API is visible; internal sources are not importable through normal package imports

3. Packaged Library Import

Use this for third-party or prebuilt libraries.

```
@co.ddap.import(library="hrlib", as="hr")
@co.ddap.import(library="paylib", as="pay")
```

Resolution:

```
library="hrlib" -> libs/hrlib.folenc, else libs/hrlib.folib
```

Only the packaged library surface is visible to the consumer.

Import Directive Fields

Field	Required	Default	Meaning
-------	----------	---------	---------

Field	Required	Default	Meaning
<code>package</code> or <code>library</code>	one required	—	logical package path or packaged library name
<code>library</code>	×	<code>false</code>	when <code>true</code> , <code>package=</code> resolves to a source library surface file
<code>expect</code>	×	inferred from library surface	expected library kind such as <code>ffi</code> , <code>system</code> , <code>advanced</code> , or <code>application</code>
<code>as</code>	×	none — full dot path required when omitted	local alias; valid FoLang identifier
<code>realm</code>	×	<code>app</code>	isolation domain
<code>parent-realm</code>	×	—	realm shadowing relationship

Notes:

- `as` is optional — when omitted, no short alias is created and the full imported package path must be used to access symbols
- dots are not allowed in `as`
- `expect` is validation, not the source of truth
- the source of truth is `@co.dap.library(type="...")` on the resolved surface file

Examples:

```
@co.ddap.import(package="hr.employee", as="emp")
emp.Employee

@co.ddap.import(package="hr.employee")
hr.employee.Employee
```

Valid `as` Values

```
as="hr"           ✓
as="v1_hr"        ✓
as="v1.hr"        ✗
as="123hr"        ✗
```

Library Surfaces

FoLang uses surface files in two distinct situations:

- Packaged library project surface**
- Application-workspace source library surface**

A surface file is always annotated with `@co.dap.library` and always represents the public contract of one library identity.

```
app.fol  -> application entry
hrlib.fol -> packaged library surface
ffi.fol  -> source library surface when imported with library=true
```

Packaged Library Project Surface

A packaged library project has no entry file. Instead it has one surface file at the project root.

Example:

```
// hrlib.fol
@co.dap.library

Employee co.lang.struct = {
    id    co.lang.int;
    name  co.lang.string;
}

IEmployee co.lang.interface = {
    getId()->(co.lang.int);
    getName()->(co.lang.string);
}

newEmployee(id co.lang.int, name co.lang.string)->(IEmployee)=>>emp.create(this, id,
name);
```

Rules:

- exactly one `@co.dap.library` file per packaged library project
- it must be at project root
- it is compiled into the packaged library artifact such as `.folib` or `.folenc`
- consumers import it with `@co.ddap.import(library="...")`

Application-Workspace Source Library Surface

A source library may also live inside an application workspace.

Example import:

```
@co.ddap.import(package="com.abc.ffi", library=true, expect="ffi", as="ffilib")
```

This resolves to a single surface file:

```
/appl/com/abc/ffi.fol
```

Rules:

- a source-library surface file must live **below** the application root, never at the application root
- multiple source libraries may exist inside one application workspace
- such a surface is imported only through `package="..."` with `library=true`
- only the surface API is importable; internal implementation sources remain hidden behind the surface
- once something is treated as a library, it does **not** expose subpackages as ordinary package imports
- this rule is the same for packaged libraries and source libraries: the surface is the boundary

General Surface Rules

These rules apply to **all** library surfaces:

- surface files expose the public contract only
- executable code is forbidden
- free functions with logic are forbidden
- classes and modules are not allowed directly in a surface unless a stricter kind-specific rule explicitly says otherwise

- structs are allowed in surface files; associated functions on those structs must follow the delegation-chain-only rule — no logic permitted

Surface File Contents — General

For **application** and **advanced** libraries, the surface may contain:

- structs
- interfaces
- signatures
- enums / ADTs
- type aliases / newtypes
- delegation-only free functions

system and **ffi** libraries use stricter surface rules defined in **Library Kinds** below.

Library Kinds

Library kinds are declared on the surface file:

```
@co.dap.library(type="ffi")
@co.dap.library(type="system")
@co.dap.library(type="advanced")
@co.dap.library(type="application")
```

Shared Meaning

- **application** -> ordinary public library surface
- **advanced** -> macro / template / concurrency / runtime machinery libraries
- **system** -> unsafe low-level runtime / pointer / process / native control libraries
- **ffi** -> foreign interface libraries

system and **ffi** Libraries

These library kinds override the general surface rules above. Their surfaces and internals are intentionally more restricted.

Internally allowed

- pointers / references / addresses / word-level types
- native functions
- structs
- free functions
- basic value-typed functions
- type aliases
- basic threading / process primitives

Internally forbidden

- classes
- modules
- interfaces
- signatures
- overloading / overriding
- new operator definitions
- generics / templates
- macros

- reflection / dynamic runtime
- dependent types / type classes
- function patterns
- closures / currying
- advanced concurrency machinery such as continuations, defer, lazy evaluation, thunks
- variadic / named / optional / default parameters

Public boundary rules

For **system** and **ffi** libraries, the public surface is restricted to free-function APIs only.

Allowed across the surface:

- free functions with simple types
- free functions with **co.lang.cstruct**

Forbidden across the surface:

- interfaces
- signatures
- classes
- modules
- pointers / references / addresses
- **co.lang.struct**

Example:

```
// driver.fol
@co.dap.library(type="system")

Point co.lang.cstruct = {
  x co.lang.int;
  y co.lang.int;
}

@co.dap.private
gpio co.lang.word->>(*);

@co.dap.public
init()->(co.lang.bool) = { ... }

@co.dap.public
getOrigin()->(Point) = { ... }
```

advanced Libraries

Allowed:

- all ordinary language features
- macros
- templates
- async / parallel / continuation machinery
- definition and configuration of concurrency / transformation behavior

Forbidden:

- pointers
- references
- addresses

- native functions

application Libraries

Allowed:

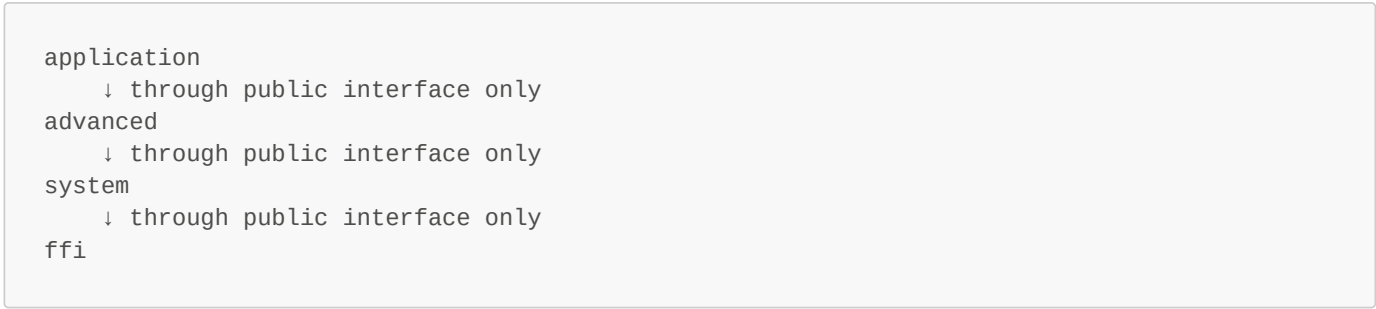
- ordinary safe language features
- public surface contracts such as structs, interfaces, signatures, enums, type aliases, and delegation-only free functions

Forbidden:

- pointers
- references
- addresses
- native functions (annotated with `@co.dap.native`)
- macros
- low-level concurrency machinery
- templates / async / parallel / continuations as defining machinery

Dependency Direction

Allowed dependency flow is one-way:



Cross-Library Communication

From	To	Allowed?	Notes
application	advanced	✓	public interface only
application	system	✓	public interface only
application	ffi	✓	public interface only
advanced	system	✓	public interface only
advanced	ffi	✓	public interface only
advanced	application	×	compiler error
system	ffi	✓	public interface only
system	application	×	compiler error
system	advanced	×	compiler error
ffi	application	×	compiler error
ffi	system	×	compiler error
ffi	advanced	×	compiler error

Realms

Realms provide import isolation, coexistence of versions, and controlled shadowing.

Realm declarations are always syntactically valid — you can always write `realm=` on any import. However realm isolation is **active only when the application entry file is annotated with `@co.ddap.dynamicruntime`**. Without it, realm declarations are accepted but silently ignored and all imports behave as a single `app` realm.

`@co.ddap.dynamicruntime` is only valid on the application entry file — not on library surface files or any other `.fol` file.

Default realm:

```
app
```

Hierarchy example:

```
core
└─ app
    ├── app1
    │   └─ app3
    └─ app2
```

Rules:

- `realm` defaults to `app`
- if `parent-realm` is omitted, parent defaults to `app`
- `parent-realm` is meaningful only when `realm` is explicitly provided and is not `app`
- libraries are always static
- `@co.ddap.dynamicruntime` is allowed only on the application entry file

Version Coexistence

```
@co.ddap.import(package="hr", realm="app", as="hr")
@co.ddap.import(package="v1.hr", realm="x", as="v1_hr")
```

Shadowing

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="a", realm="x", parent-realm="app", as="hr")
```

When the alias is the same and the child realm points to the parent realm, the child shadows the parent.

Import Binding Rules

Invalid

Same alias, same realm, different package:

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="b", realm="app", as="hr") // error
```

Valid aggregation

```
@co.ddap.import(package="folderx.some", realm="app", as="hr")
@co.ddap.import(package="folderx.some", realm="app", as="hr")
```

Valid coexistence

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="a", realm="x", as="v1_hr")
```

Valid shadowing

```
@co.ddap.import(package="a", realm="app", as="hr")
@co.ddap.import(package="a", realm="x", parent-realm="app", as="hr")
```

Cycles

Compiler error if any cycle exists through:

- package imports
- realm parent relationships

Examples:

- `packageA` imports `packageB`, and `packageB` imports `packageA`
- `realm="x", parent-realm="y"` and another import uses `realm="y", parent-realm="x"`

Symbol Resolution

Resolution Order

For symbols without a prefix:

1. current package symbol table
2. parent package chain upward
3. error if not found

For symbols with an alias prefix:

1. current package imported-context map
2. parent package chain imported-context maps
3. error if not found

For imports declared **without** `as`:

1. use the full imported package path directly
2. resolve it through imported-context lookup using that full path
3. error if not found

Example Context Graph

```
pp1 (grandparent package)
├── SymbolTable
├── ImportedContexts
└── ChildCtxs
```

```

p1 (parent package)
├─ Parent -> &p1
├─ SymbolTable
├─ ImportedContexts
└─ ChildCtxs

c1 (current package)
├─ Parent -> &p1
├─ SymbolTable
├─ ImportedContexts
└─ ChildCtxs

```

Resolution Examples

```

lookup "OwnType"
  -> current symbol table

lookup "ParentType"
  -> parent chain

lookup "hr.Employee"
  -> imported-context lookup for alias "hr"

lookup "hr.employee.Employee"
  -> imported-context lookup for full imported package path when no alias exists

lookup "unknown.Type"
  -> imported-context lookup fails -> compiler error

```

Short Summary

- folders define packages
- root is never a package
- `co.*` is always available
- `@co.ddap.alias` only shortens `co.*`
- `@co.ddap.import(package="...")` imports normal packages
- `@co.ddap.import(package="...", library=true, ...)` imports same-owner source libraries
- `@co.ddap.import(library="...")` imports packaged external libraries
- `expect="..."` is an import-site assertion, not the source of truth
- `@co.dap.library(type="...")` is the source of truth for library kind

Operators

Arithmetic operators

`+, -, *, /, %, **, ++, --`

Logical operators

`&&, ||, !, &, |`

Comparison operators

`==, !=, <, >, <=, >=`

Other operators

Pointer Declaration

```
somePtr    co.lang.int->>(*);
someDb1Ptr co.lang.int->(**);
```

Array Declaration

```
someArray      co.lang.int->([5]);
someDb1Array   co.lang.int->([2,3]);
someJaggedArray co.lang.int->([2][3]);
someVLArray    co.lang.int->([...]);
someZeroLA     co.lang.int->([0]);
someZeroDimA   co.lang.int->([.]);
```

Array Declaration with Initialization

```
someInitializedArray  co.lang.int->([3]) = [1, 2, 3];
someInitializedArray1 co.lang.int->([ ]) = [1, 2, 3];
someInitializedDb1Array co.lang.int->([,]) = [[1, 2], [3, 4]];
```

Reference Declaration

```
someRef      co.lang.int->(&);    // reference
someLValueRef co.lang.int->(&&);  // LValue reference
someHpRef     co.lang.int->(~);    // heap allocated reference
someAddr      co.lang.int->(@);   // address
someThunk     co.lang.int->(^);   // thunk
someSlice     co.lang.int->([:]); // slice
```

Range Declaration

```
// Typed range variable declaration
someRange co.lang.int->(..);

// Inferred range declarations
rangeI := 1..10;      // [1, 10]  ExcludeStart=false, ExcludeEnd=false
rangeS := 0<..5;      // (0, 5]   ExcludeStart=true,  ExcludeEnd=false
rangeL := 0..<100;    // [0, 100) ExcludeStart=false, ExcludeEnd=true
rangeB := 0<..<100;   // (0, 100) ExcludeStart=true,  ExcludeEnd=true
rangeE := ..100;      // open lower bound  (_, 100]
rangeF := 1..;        // open upper bound  [1, _)
```

Auto and Dynamic Variable Declaration

```
someAutoVar    co.lang.auto    = "Hello"; // type inferred from value; initialization
required
someDynamicVar co.lang.dynamic;           // dynamic typing
```

Values

```
someVar co.lang.data = 10; // initialization required
```

Bind Variables

`$_[0-9]*`

Discard / Wildcard Variable

Comma and Grouping

```
// Comma
x co.lang.int = 10, y co.lang.string = "Hello", z co.lang.bool = co.const.true;

// Grouping
(x co.lang.int = 10, y co.lang.string = "Hello", z co.lang.bool = co.const.true);
```

Fat Pointers

```
x co.lang.int->>(*, kind="", meta={});

co.lang.int->>(*, meta={});

y co.lang.int->>(*, meta={len:co.lang.usize, vtab:somepkg.VTable->(*)})

z co.lang.int->>(* ,kind=region, meta={})
```

```
Pointer
├─ base_type: T
├─ kind: <FatKind>
│   ├── thin
│   ├── slice
│   ├── relative
│   ├── trait
│   ├── buffer
│   ├── view
│   ├── opaque
│   ├── custom
│   ├── mem
│   ├── nullptr
│   ├── sptr
│   ├── uptr
│   ├── ptrdiff
│   ├── usize
│   ├── ssize
│   └─ (region) ← optional syntactic sugar
└─ meta:
    ├── region: heap | stack | global | numa(N) | mmio | constant | ...
    └─ len, cap, vtab, bits, endian, ...
```

Pointers for address manipulation

```
y co.lang.word->(repr=intptr);
z co.lang.word->(sign=unsigned, repr=uintptr);
p co.lang.word->(repr=ptrdiff);
n co.lang.word->(sign=unsigned, repr=usize);
m co.lang.word->(repr=isize);
o co.lang.void->(repr=nullptr);
```

Relative Pointers

```
z co.lang.int->(*,kind=relative, meta={})
```

Types and Kinds

```
x co.lang.int = 10;
x.type() → co.lang.int
x.kind() → co.lang.nothing

x co.lang.data = 10;
x.type()      → co.lang.value
x.kind()      → co.lang.data
x.type().type() → co.lang.int    // to get the actual type

x co.lang.auto = 10;
x.type() → co.lang.int    // inferred at compile time, static
x.kind() → co.lang.data

x co.lang.dynamic = 10;
x.type() → co.lang.int    // can vary at runtime
x.kind() → co.lang.data
```

Type Declarations

```
// Alias
x co.lang.type = co.lang.int;

// New
x co.lang.newtype = co.lang.int;

// Opaque
x co.lang.opaquetype = co.lang.int;

// ADT (tagged union)
y co.lang.type = co.lang.int | co.lang.char;

// Subtype / covariant
test co.lang.subtype = co.lang.int;

// Supertype / contravariant
test co.lang.supertype = co.lang.int;
```

Dependent Types

Type Constructors — Functions That Return Types

A type constructor is a function that takes a value or type and returns a type. The returned type depends on the input value — this is what makes it a dependent type.

```
// Vector – type constructor function
// takes → co.lang.int (size)
// returns → co.lang.dependentType (a type)
Vector(n co.lang.int)->(co.lang.dependentType) =
  co.lang.int->([n]);

// calling Vector(3) returns a TYPE at compile time
// that type is co.lang.int->([3])
v3 Vector(3) = [1, 2, 3]; // type is Vector(3)
v4 Vector(4) = [1, 2, 3, 4]; // type is Vector(4) – different type!

// Vector(3) ≠ Vector(4) – completely different types
// size is part of the type – compiler knows at compile time
```

Type Constructor Is A Function

```
Vector      → function (type constructor)
Vector(3)   → function call → returns type co.lang.int->([3])
Vector(4)   → function call → returns type co.lang.int->([4])

just like:
  add(1, 2) → returns a value (3)
  Vector(3) → returns a type (int[3])
```

Compiler Enforced Size Safety

```
// dot product – only valid for same size vectors
// compiler enforces this via dependent types
dotProduct(a Vector(n), b Vector(n))->(co.lang.int) = {
  // n is same for both – compiler verified
}

v3 Vector(3) = [1, 2, 3];
v4 Vector(4) = [1, 2, 3, 4];

dotProduct(v3, v3) // ✓ same type Vector(3)
dotProduct(v3, v4) // ✗ compiler error – Vector(3) ≠ Vector(4)
```

Matrix — Two Parameter Type Constructor

```
// Matrix – takes rows and cols, returns dependent type
Matrix(r co.lang.int, c co.lang.int)->(co.lang.dependentType) =
  co.lang.int->([r, c]);

m34 Matrix(3, 4) = [[1,2,3,4],[5,6,7,8],[9,10,11,12]];
m45 Matrix(4, 5) = ...;
```

```
// matrix multiply – cols of A must equal rows of B
// n must match – compiler verified
multiply(a Matrix(r, n), b Matrix(n, c))->(Matrix(r, c)) = {
  // compiler ensures dimensions are compatible
}

multiply(m34, m45)    // ✓ Matrix(3,4) × Matrix(4,5) = Matrix(3,5)
multiply(m34, m34)    // ✗ compiler error – 4 ≠ 3
```

Stack — Value and Type Parameter

```
// Stack – takes size and element type
Stack(n co.lang.int, T co.lang.type)->(co.lang.dependentType) =
  T->([n]);

s Stack(10, co.lang.int)    = ...; // stack of max 10 ints
t Stack(5,  co.lang.string) = ...; // stack of max 5 strings
```

Type Is Value + Kind Combined

```
Vector(3):
  kind  = Vector    (what it is)
  value = 3         (how many)
  type  = Vector(3) (both together – the dependent type)

Vector(3) ≠ Vector(4)  ← different types entirely
Vector(3) = Vector(3)  ← same type
```

Connects to Type Constructor in Spec

```
// Option – type constructor for ADT
@co.dap.hokrt
Option(T) co.lang.type = Some(T) | None();

// Vector – type constructor for dependent type
Vector(n co.lang.int)->(co.lang.dependentType) =
  co.lang.int->([n]);

// same concept:
//   Option(T) takes a type → returns ADT type
//   Vector(n) takes a value → returns dependent type
```

Simple Dependent Type

```
identity(x co.lang.int)->(x.type) = x
```

Types Computed from Runtime Values

```

someType := somefun(value)

somefun(value co.lang.int)->(co.lang.type) = {
    (value < 100).return(co.lang.string).otherwise.return(co.lang.bool);
}

// or with annotation
@co.dap.typefromvalue
somefun(value co.lang.int)->(co.lang.type)={
    (value < 100).return("hello").otherwise.return(co.const.true);
}

// compile-time eager
@co.dap.comptime
@co.dap.eager
chooseType(value co.lang.int)->(co.lang.type) = {
    (value < 100).return(co.lang.string).otherwise.return(co.lang.bool);
}

// tagged value
somefun(value co.lang.int)->(co.lang.tag) = {
    (b < 100).return(co.lang.tag(co.lang.string, "Hello"))
    .otherwise.return(co.lang.tag(co.lang.bool, co.const.true));
}

```

Types**The three axes — each adds one new power:****Axis 1: Polymorphism (terms depend on types)**

```

// "Give me a type, I'll give you a value"

// Without: write separate functions
identityInt(x int) → int
identityStr(x string) → string

// With: one function works for all types
identity(T)(x T) → T

// This is generics / parametric polymorphism
// System F, Java generics, your @co.dap.generic

```

Axis 2: Type operators (types depend on types)

```

// "Give me a type, I'll give you a type"

List(Int)      → List of Int      // type → type
Map(String, Int) → Map            // type → type → type
Option(T)      → Some(T) | None   // type constructor

// This is kinds / higher-kinded types
// Your folang: Option(T) co.lang.type = Some(T) | None()

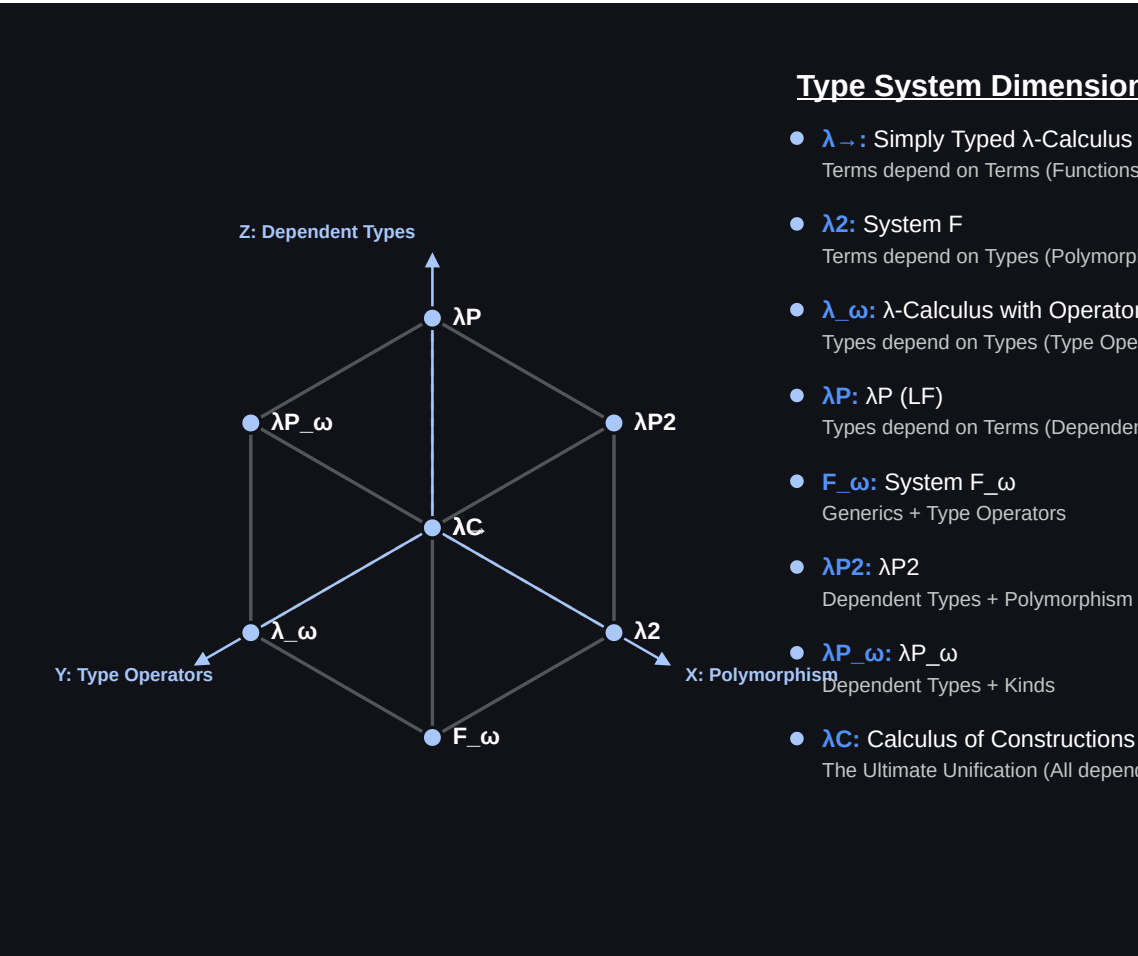
```

Axis 3: Dependent types (types depend on values)

```
// "Give me a value, I'll give you a type"

Vector(3)      → array of exactly 3 elements
Matrix(2, 3)   → 2x3 matrix
NonZero(n)     → proof that n ≠ 0

// The type changes based on a runtime value
divide(a int, b NonZero(int)) → int
// compiler PROVES b can't be zero
```



Data Structures

Struct Declaration

```
myStruct co.lang.struct={
  field1 co.lang.int;
  field2 co.lang.string;
  field3 co.lang.bool;
}
```

Struct Rules

```
structs cannot extend other structs
structs cannot have methods
structs can compose other structs
structs cannot have default values to fields/members
structs can embed other structs (Go lang like)
structs can have associated functions (Go lang like)
```

Struct Embedding

Embedding promotes fields of an embedded struct directly into the outer struct — they act as the outer struct's own fields at construction and access sites. This is distinct from composition where the embedded struct is a named field.

```
E co.lang.struct = {
    id    co.lang.int;
    name  co.lang.string;
}

// ✓ No conflict – id and name promoted as B's own fields
B co.lang.struct = {
    age  co.lang.float;
    E;           // embedded – id and name promoted
}

b := B{ age: 25.0, id: 1, name: "Rao" }; // all fields at same level
b.id    // direct – no b.E.id needed
b.name  // direct – no b.E.name needed
b.age   // direct
```

```
// ✗ Compiler error – name conflict between B.name and E.name
B co.lang.struct = {
    name co.lang.string; // conflicts with E.name
    E;
    age  co.lang.float;
}
// Fix 1 – rename B's conflicting field
// Fix 2 – use explicit composition instead: e E;
```

```
// Explicit composition – no promotion, always qualified access
B co.lang.struct = {
    name co.lang.string;
    e    E;           // named field – no conflict, no promotion
    age  co.lang.float;
}

b.name    // B's own name
b.e.id    // E's id – always explicit
b.e.name  // E's name – always explicit
```

Embedding Rules

Situation	Behavior
Embedded field, no conflict	Promoted — acts as the outer struct's own field

Situation	Behavior
Embedded field, name conflict with outer	× Compiler error — rename or use composition
Multiple embeds, no conflict between them	All fields promoted
Multiple embeds, conflict between embedded structs	× Compiler error
Explicit composition (e E)	No promotion — always accessed via <code>b.e.field</code>

FoLang does **not** silently shadow conflicting fields. Any name conflict is a compiler error — the programmer must make a conscious decision to rename or switch to explicit composition.

Struct Inner Type Rules

```
structs can declare inner structs           ✓ data composing data
structs can declare inner enums/ADTs        ✓ data variant – natural
structs cannot declare inner classes         ✗ compiler error – struct is pure data
structs cannot declare inner modules         ✗ compiler error – struct is pure data

// ✓ valid – struct declaring inner struct
Employee co.lang.struct = {
  Address co.lang.struct = {
    street co.lang.string;
    city   co.lang.string;
  }
  id      co.lang.int;
  name    co.lang.string;
  address Address;
}

// ✓ valid – struct declaring inner enum
Employee co.lang.struct = {
  Status co.lang.enum = {
    Active,
    Inactive
  }
  id      co.lang.int;
  status Status;
}

// ✗ compiler error – struct declaring inner class
Employee co.lang.struct = {
  Validator co.lang.class = {    // ✗ struct is pure data
    validate()->(co.lang.bool) = { ... }
  }
  id co.lang.int;
}
```

C-Struct Declaration

`co.lang.cstruct` is a C-like value type — passed by value, simple memory layout, safe to cross zone boundaries. Unlike `co.lang.struct` which is passed by reference, `co.lang.cstruct` is always copied on pass.

```
Point co.lang.cstruct = {
  x co.lang.int;
  y co.lang.int;
}

Rect co.lang.cstruct = {
```

```

    origin Point;
    width  co.lang.int;
    height co.lang.int;
}

```

cstruct Rules

always passed by value – never by reference
 simple memory layout – no metadata
 can contain only simple types and other cstructs
 cannot contain co.lang.struct ✗ has metadata
 cannot contain co.lang.string ✗ heap allocated
 cannot contain co.lang.dynamic ✗ runtime type info
 cannot contain classes ✗ vtable, metadata
 cannot contain modules ✗
 cannot contain any heap allocated type ✗
 cannot have methods
 cannot have associated functions
 cannot embed co.lang.struct
 safe to cross zone boundaries in public interface

allowed field types:

co.lang.int, co.lang.uint, co.lang.float	✓	primitives
co.lang.bool, co.lang.char, co.lang.byte	✓	primitives
co.lang.int->([N])	✓	fixed size arrays
co.lang.cstruct	✓	other cstructs

Packed cstruct — no padding, exact memory layout

Used for hardware registers, binary protocols, exact memory mapped formats:

```

@co.dap.packed
Register co.lang.cstruct = {
    flags  co.lang.uint8;
    status co.lang.uint8;
    data   co.lang.uint16;
}

```

SIMD cstruct — aligned for vector operations

Used for math, graphics, signal processing:

```

@co.dap.simd(align=16)
Vec4 co.lang.cstruct = {
    x co.lang.float;
    y co.lang.float;
    z co.lang.float;
    w co.lang.float;
}

```

Both together

```
@co.dap.packed
@co.dap.simd(align=32)
AVXVec co.lang.cstruct = {
    data co.lang.float;
}
```

`@co.dap.packed` and `@co.dap.simd` are specialisations of `co.lang.cstruct` — same rules, same zone boundary safety. They are not separate types.

Enum Declaration

```
myEnum co.lang.enum={
    Variant1,
    Variant2,
    Variant3
}
```

Union Declaration

```
myUnion co.lang.union={
    intValue co.lang.int;
    strValue co.lang.string;
}
```

Classes

```
Employee co.lang.class ={
    getEmployeeDetails()->(Employee) = empmodule.getEmployeeDetails;
    // assigning module function to class's method

    getEmployeeInfo()->(Employee) ==> empmodule.getEmployeeDetails();
    // delegating – internally redirecting the call to module function
}

// $1, $2, $3 ... are previous results captured as bind variables
Emp co.lang.class={
    dosomething(a co.lang.int, b co.lang.int)->
    (co.lang.int)==>somePack.someMethod(a)==>someOthPack.someOtherMeth($1, b);
}
```

Inner Type Declaration Rules

Outer → Inner	Allowed?	Reason
class → struct	✓	data scoped to class — natural
class → class	✓	natural OOP nesting
class → enum/ADT	✓	variant scoped to class
class → module	×	compiler error — module is standalone
struct → struct	✓	data composing data

Outer → Inner	Allowed?	Reason
struct → enum/ADT	✓	data variant — natural
struct → class	×	compiler error — struct is pure data
struct → module	×	compiler error — struct is pure data
module → struct	✓	data types for module functions
module → enum/ADT	✓	variants for module functions
module → class	×	compiler error — module has no instances
module → module	×	compiler error — use package nesting instead

```
Employee co.lang.class = {  
  
  // ✓ inner struct – scoped to Employee  
  Address co.lang.struct = {  
    street co.lang.string;  
    city   co.lang.string;  
  }  
  
  // ✓ inner ADT – scoped to Employee  
  Status co.lang.type = Active | Inactive | Pending;  
  
  address Address;  
  status  Status;  
  
  @co.dap.method.instance  
  getAddress()->(Address) = { ... }  
}  
  
// accessing inner types from outside  
a Employee.Address = Employee.Address{ street: "MG Road", city: "Mumbai" }  
s Employee.Status  = Employee.Status.Active
```

```
Inner types follow the same access rules as methods – `@co.dap.private`, `@co.dap.public`  
etc. apply.  
  
### Method Types  
  
```folang  
Employee co.lang.class = {

 @co.dap.method.static
 getEmployee()->(Employee) = {}

 @co.dap.method.instance
 getEmployee()->(Employee) = {}

 @co.dap.method.class
 getEmployee()->(Employee) = {}

 @co.dap.method.object
 getEmployee()->(Employee) = {}
}

@co.dap.oops(

```

```

A: { inherit:true, virtual:true },
B: { implements:true },
C: { inherits:true, abstract=true },
D: { inherits:true },
E: { uses:true },
F: { composes:true },
G: { extends:true },
H: { with:true }
)
test co.lang.class->(uses=[], implements=[], extends=[], inherits=[], with=package.type,
composes=[]) = {
 getTest(id int)->(test) = {}
}

```

## The @@new and @@init Methods

`@@new` and `@@init` are lifecycle methods — compiler-owned, not user-definable outside the class. `@@` signals they are restricted lifecycle symbols, not regular methods.

```

@co.dap.generic(type={T:{typename}, R:{typename}})
Employee co.lang.class = {

 id T
 name R

 // @@new is provided by default even if not overridden.
 // Override only when you genuinely need to change allocation behavior.

 @co.dap.method.class
 @co.dap.private
 @@new()->(co.lang.uninit) = { self.return co.const.none }

 @co.dap.method.class
 @co.dap.public
 @@new(a co.lang.typevalue, b co.lang.typevalue)->(co.lang.uninit) = {
 // Manual type aliasing – @co.dap.generic handles this automatically
 // Override @@new only when you need custom allocation logic
 T co.lang.type = a
 R co.lang.type = b

 // self keyword is allowed only in class methods
 self.parent.@@new();

 // uninit instance method internally calls @@new and @@init
 self.return co.lang.uninit.instance(Employee, self);
 }

 @co.dap.override
 @co.dap.constructor(access=private)
 @@init() = {}

 @co.dap.override
 @co.dap.constructor(access=public)
 @@init(id T, name R) = {
 this.parent.@@init();
 this.id = id;
 this.name = name;
 }

 getEmployee(id T)->(Employee) = {}
}

```

## Module Declaration

```
EmployeeModule co.lang.signature={
 // module contents
 getEmployee(id co.lang.int)->(Employee);
}

@co.dap.module(signature=EmployeeModule)
EmployeeModImpl co.lang.module->(signature=EmployeeModule, matches=EmployeeModule) = {

 Employee co.lang.struct={
 Id co.lang.int;
 Name co.lang.string;
 }

 getEmployee(id co.lang.int)->(Employee)={
 this.return Employee{
 Id:10, Name:"Rao",
 };
 }
}

mm EmployeeModule = EmployeeModuleImpl;
v mm.Employee = mm.Employee{Id:10, Name:"Rao"};
mm.getEmployee(10);
```

### Module Inner Type Rules

```
// ✓ valid – module declaring inner struct
EmployeeModule co.lang.module = {

 Config co.lang.struct = {
 timeout co.lang.int;
 retries co.lang.int;
 }

 Status co.lang.enum = {
 Active,
 Inactive
 }

 connect(cfg Config)->(co.lang.bool) = { ... }
 getStatus()->(Status) = { ... }
}

// ✗ compiler error – module declaring inner class
EmployeeModule co.lang.module = {
 Validator co.lang.class = { // ✗ module has no instances
 validate()->(co.lang.bool) = { ... }
 }
}

// ✗ compiler error – module declaring inner module
EmployeeModule co.lang.module = {
```

```
HelperModule co.lang.module = { } // ✗ use package nesting instead
}
```

## Functions

### Normal

```
fun1 (k co.lang.int, b co.lang.char)->(co.lang.int, co.lang.char)={
 // function body
}
```

### Local Type Declarations

Functions can declare types locally. Local types are scoped to the function body only — they cannot appear in the function's parameter or return types, and are not accessible outside.

```
processEmployee()->(co.lang.bool) = {

 // Local ADT – scoped to this function only
 Status co.lang.type = Active | Inactive | Pending;

 // Local struct – scoped to this function only
 LocalRecord co.lang.struct = {
 id co.lang.int;
 status Status;
 }

 r := LocalRecord{ id: 1, status: Active };
 this.return r.status == Active;
}
```

```
// ✗ Compiler error – local type cannot appear in return type
getRecord()->(LocalRecord) = {
 LocalRecord co.lang.struct = { id co.lang.int; }
 this.return LocalRecord{ id: 1 };
}
```

Local types can be passed to inner functions defined within the same scope:

```
process()->(co.lang.int) = {
 Status co.lang.type = Active | Inactive;

 // Inner function – can use Status because it shares the same scope
 check(s Status)->(co.lang.bool) = {
 this.return s == Active;
 }

 this.return check(Active).return(1).otherwise.return(0);
}
```

### Curried

```
add(first co.lang.int)(second co.lang.int)->(co.lang.int)={
 this.return first + second
}
```

## Closure

```
add() -> ((co.lang.int) -> co.lang.int) = {
 sum co.lang.int = 0
 this.return (x co.lang.int) -> (co.lang.int) = {
 sum += x
 this.return sum
 }
}
```

## Functions Taking and Returning Functions

### Syntax 1 — Inline signature

```
someFunction (r (co.lang.int, co.lang.int)->(co.lang.int))->((co.lang.int)->
(co.lang.int))={}
```

### Syntax 2 — Named type alias

```
someFArg co.lang.type = (co.lang.int, co.lang.int)->(co.lang.int)
someFRet co.lang.type = (co.lang.int)->(co.lang.int)

someFunction (r someFArg)->(someFRet)={}
```

### Syntax 3 — Function objects

```
someFArg co.lang.function = (a co.lang.int, b co.lang.int) -> (co.lang.int)={
 this.return a + b;
}

someFRet co.lang.function = (a co.lang.int) -> (co.lang.int)={
 this.return a * 2;
}

someFunction (r someFArg)->(someFRet)={}
```

## Anonymous Functions and Objects

### Anonymous Classes/Types

```
emp := co.lang.class{};

empObj := emp.new();

empobj1 := co.lang.class{
```

```
 name string
 }.new();
```

## Anonymous Functions

```
add := (a int, b int) -> (int) {
 this.return a + b;
};

res := (a int, b int) -> (int) {
 this.return a * b;
})(10, 20);
```

## Lambda

Only allowed as an inline callback argument to collection operations (e.g. `map`, `filter`, `reduce`, `forEach`, `sortBy`, `groupBy`). Using `|...|` anywhere else is a syntax/lint error.

```
// Syntax
|x, y| => x + y;

// Collection use – allowed
nums.map(|x| => x*x)
words.filter(|s| => s.len() > 3)
pairs.reduce(|acc, e| => acc + e, 0)
dict.map(|k, v| => v * 10)
list.sortBy(|a, b| => a.score - b.score)
```

## Inner Function

```
myfun(a co.lang.int, b co.lang.int)->(co.lang.int)={
 p co.lang.int = 10;
 someother()->()={
 co.out.println(p);
 }
 someother();
 p = 20;
 someother();
}
```

## Function Objects

```
myobj co.lang.function = (a co.lang.int, b co.lang.int)->(co.lang.int)={
 this.return a + b;
}

add (a co.lang.int, b co.lang.int)->(co.lang.int){ this.return a + b; }
oObj co.lang.function = add;

funtype co.lang.type = (a co.lang.int, b co.lang.int)->(co.lang.int);

closure(factor int) => (x int) = x * factor;
```

```
curry(factor int)(val int) = factory * val;
```

## Default Parameters

```
fun1 (k co.lang.int, b co.lang.char = 10)->(co.lang.int, co.lang.char)={
}
```

## Variadic Functions

Curried functions are not allowed to be variadic, and vice versa.

```
fun1 (k co.lang.int, ...b co.lang.char)->(co.lang.int, co.lang.char)={
}
```

## Optional Parameters

```
fun1(k? co.lang.int)->()={
 if k.omitted{

 }else{

 }
}
```

## Named Parameters

```
fun1(~k co.lang.int)->()={

}
```

## Named Returns

```
doManythings(a co.lang.int, b co.lang.int->(&, meta={type=out}))->(r co.lang.int, e
co.lang.exception)={}
```

## Function Delegates

```
@co.dap.delegate someDelegate co.lang.delegate = (a co.lang.int, b co.lang.int) ->
(co.lang.int, co.lang.int);
```

## Function Chaining

```
fetchEmployee(empId co.lang.string)->(Employee)=>>empMod.getEmployee(this, empId);
```

## Associated Functions

```
(emp empMod.Employee) fetchEmployee(empId co.lang.string)->
(empMod.Employee)==>>empMod.getEmployee(emp, empId);
```

## Scoping Rules for Functions

All functions in FoLang have a defined scope — the set of variables a function can access.

---

### Default — Lexical / Static Scope

All of the following are **always** lexically scoped — this cannot be changed:

```
methods
inner methods
free functions
inner functions
closures
lambdas
Generic Functions
Anonymous functions
Curried functions
```

Lexical scope means the function can access variables defined at its **declaration site** — not its call site.

```
x co.lang.int = 10;

// lexically scoped – can access x at declaration site
foo()->() = {
 co.out.println(x); // ✓ x from declaration scope
}
```

---

### Associated Functions — Additional Scope Options

Only associated functions support non-lexical scoping via annotations:

**@co.dap.fun.lexicalscope** — default, explicit declaration

```
@co.dap.fun.lexicalscope
(emp Employee) process()->() = {
 co.out.println(emp.name); // ✓ declaration scope
}
```

**@co.dap.fun.dynamicscope** — accesses caller's scope

```
@co.dap.fun.dynamicscope
(emp Employee) process()->() = {
 co.out.println(name); // name comes from caller's scope
}
```

**@co.dap.fun.mixedscope** — accesses both scopes

```
// caller scope takes priority – shadows declaration scope on conflict
@co.dap.fun.mixedscope
(emp Employee) process()->() = {
 co.out.println(name); // caller scope takes priority
 co.out.println(emp.id); // falls back to declaration scope
}
```

### Lambda Scope Inside Dynamically Scoped Methods

A lambda is always lexically scoped — this never changes. However when a lambda is passed to a dynamically scoped associated method such as **reduce**, **map**, **filter**, or **each**, it executes within that method's dynamic scope context. The caller's variables become accessible through the method's scope, not through the lambda itself.

```
nums.reduce(|acc, e| => {
 total += e // accessible via reduce's dynamic scope – not the lambda's own scope
 acc + e
}, 0)
```

The lambda did not change. **reduce** is a dynamically scoped associated method — it carries the caller's scope and the lambda executes inside it.

### Why Dynamic Scope Exists — .do .loop .each and Collections

The entire FoLang control flow model is built on dynamically scoped associated functions. Without dynamic scope these constructs cannot read or modify the caller's variables — making them useless:

```
x co.lang.int = 10;
total co.lang.int = 0;
arr co.lang.int->([5]) = [1, 2, 3, 4, 5];

// .do – dynamic scope – reads and modifies caller's x
(x > 5).do({
 x = 20 // ✓ modifies caller's x
 co.out.println(x) // ✓ reads caller's x
})

// .loop – dynamic scope – modifies caller's x
(x > 0).loop({
 x-- // ✓ caller's x
})

// .each – dynamic scope – modifies caller's total
arr.each(_, val).do({
 total += val // ✓ caller's total
})

// .filter .map .reduce – dynamic scope
nums.filter(|x| => x > 10)
nums.reduce(|acc, e| => {
 total += e // ✓ caller's total
 acc + e
}, 0)
```

If `.do` were lexically scoped:

```
do's declaration site has no x
x would be invisible ✕
cannot read caller's x ✕
cannot modify caller's x ✕
control flow becomes useless ✕
```

FoLang Control Flow Is Dynamic Scope

```
no if/else keywords → .do / .otherwise – dynamic scope
no for/while keywords → .loop – dynamic scope
no foreach keywords → .each – dynamic scope
no in keywords → .contains – dynamic scope
no map/filter keywords → .map / .filter – dynamic scope

all control flow implemented as associated functions
all dynamically scoped
caller variables fully accessible and modifiable
no special language constructs needed
scope model directly enables control flow model
```

Conflict Resolution in Dynamic and/or Mixed Scope

```
same variable name exists in both scopes:
 Local scope → wins – shadows Caller scope
 caller scope → wins – shadows declaration scope
 declaration scope → masked for that variable
```

Additional Rules for non lexical scopes for associated functions

```
1. Non lexical scope functions cannot be passed or returned
```

Scope Rules Summary

Function Type	Lexical	Dynamic	Mixed
methods	✓ only	×	×
inner methods	✓ only	×	×
free functions	✓ only	×	×
inner functions	✓ only	×	×
anonymous functions	✓ only	×	×
closures	✓ only	×	×

Function Type	Lexical	Dynamic	Mixed
lambdas	✓ only	×	×
associated functions	✓ default	✓ opt-in	✓ opt-in
<code>.do / .loop / .each</code>	×	✓ built-in	×
<code>.map / .filter / .reduce</code>	×	✓ built-in	×

Indexer

```
MyList co.lang.struct = {
 eles co.lang.int->([]*);
}

@co.dap.indexer(symbol="[]")
(g MyList) get(index co.lang.int)->(co.lang.int) = {
 this.return g.eles[index]
}

@co.dap.indexer(symbol="[]=")
(g MyList) set(index co.lang.int, value co.lang.int)->() = {
 g.ele[index] = value
}
```

Inline

```
@co.dap.inline
add(a co.lang.int, b co.lang.int)->(co.lang.int) = {
 this.return a + b;
}
```

Lazy

```
@co.dap.lazy
x = add(1, 2);
```

Native Functions

```
@co.dap.native
nativeMethod(a co.lang.int, b co.lang.int)->(co.lang.int) = {
 // native implementation
}
```

---

Generics

```
@co.dap.generic(
 at=runtime,
 type={
 T: {variance:invariant, bound=Number, kind:param, impredicative:false},
 R: {variance:invariant, bound=Number, kind:return}
```

```
 }
)
 add(a T, b T)->(R) = { this.return a + b; }
```

Generic annotation fields:

- `at` — `runtime` | `compiletime` (acts like C++ templates)
- `refied` — `true` | `false`
- `where` — `usesite` | `callsite`

typename/type attributes:

Attribute	Values
variance	<code>covariant</code> , <code>invariant</code> , <code>contravariant</code>
bound	type to bind
kind	<code>param</code> , <code>result</code> , <code>var</code> , <code>arg</code>
default	default type
nullable	bool
inclusive	bool
impredicative	bool — when <code>true</code> , allows <code>T</code> to be instantiated with a <code>forall</code> type (v2)
typekind	<code>type</code> , <code>class</code> , <code>function</code> , <code>module</code> , <code>package</code>
types	list of allowed types for constraints

Generic Functions — Parameters and Return Values

Rank-1: Outer function is generic; parameter uses the same type variable

`T` is fixed at the call site before the function parameter is used. The passed function is already monomorphic inside the body.

Syntax 1 — Inline signature

```
@co.dap.generic(type={T:{variance:invariant}})
someFunction(f (T, T)->(T), a T)->(T) = {}
```

Syntax 2 — Named type alias

```
@co.dap.generic(type={T:{variance:invariant}})
someFArg co.lang.type = (T, T)->(T)

someFunction(f someFArg, a T, b T)->(T) = {}
```

Syntax 3 — Function objects

```
someFArg co.lang.function = (a co.lang.int, b co.lang.int)->(co.lang.int) = {
 this.return a + b;
}

@co.dap.generic(type={T:{variance:invariant}})
someFunction(f someFArg, a T)->(T) = {}
```

## Rank-2: The function parameter is itself polymorphic (higher-rank)

The passed function stays generic **inside the callee**. The callee decides what **T** is. Uses existing **forall**.

### Syntax 1 — Inline signature

```
someFunction(f forall(T).(T, T)->(T))->(co.lang.int) = {
 this.return f(1, 2);
}
```

### Syntax 2 — Named type alias

```
someFArg co.lang.type = forall(T).(T, T)->(T)

someFunction(f someFArg)->(co.lang.int) = {}
```

**Syntax 3 — Function objects** × Compiler error. **co.lang.function** is a concrete value — it holds a monomorphic function instance and cannot carry **forall**. There is no need for a separate **co.lang.polyfunction** kind either; **forall(T).(T)->(T)** is already a type, and **co.lang.type** is the correct and sufficient container for it. Use Syntax 1 or Syntax 2 for Rank-2:

```
// Correct — Syntax 2 with co.lang.type
someFArg co.lang.type = forall(T).(T, T)->(T)

someFunction(f someFArg)->(co.lang.int) = {}
```

## Returning Generic Functions

### Rank-1 return

```
@co.dap.generic(type={T:{variance:invariant}})
makeAdder(a T)->((T)->(T)) = {
 this.return (b T)->(T) = { this.return a + b; };
}
```

### Rank-2 return — returning a polymorphic function

```
makeIdentity()->(forall(T).(T)->(T)) = {
 this.return forall(T).(x T)->(T) = { this.return x; };
}
```

## Rank-3: A Parameter is Itself a Rank-2 Function

Rank-3 works naturally in FoLang via **forall** nesting. No new constructs needed.

### Syntax 1 — Inline

```
// f takes a Rank-2 function as its argument – that is Rank-3
applyRank2(
 f (forall(T).(T, T)->(T)) -> (co.lang.int)
) -> (co.lang.int) = {
 this.return f(1, 1);
}
```

## Syntax 2 — Named type aliases (cleaner)

```
rank2FnType co.lang.type = forall(T).(T, T)->(T)
rank3ArgType co.lang.type = (rank2FnType) -> (co.lang.int)

applyRank2(f rank3ArgType) -> (co.lang.int) = {
 this.return f(1, 1);
}
```

## Rank-3 return

```
makeRank2Consumer() -> ((forall(T).(T)->(T)) -> (co.lang.int)) = {
 this.return (f forall(T).(T)->(T)) -> (co.lang.int) = {
 this.return f(42);
 };
}
```

## Impredicativity — Instantiating **T** with a **forall** Type

Impredicativity is when a type variable **T** in a generic is itself instantiated with a **forall** type. Example of what this means:

```
@co.dap.generic(type={T:{variance:invariant}})
box(x T) -> (Box(T)) = {}

// Impredicative call – T being set to forall(U).(U)->(U)
result := box(forall(U).(U)->(U)); // ✗ not legal without explicit opt-in
```

Most type systems reject this by default. FoLang takes an opt-in approach.

### v1 Workaround — Option C: Wrapping with **co.lang.type**

Not true impredicativity but solves 90% of practical cases:

```
polyId co.lang.type = forall(U).(U)->(U)

// box takes co.lang.type – no impredicative unification needed
box(x co.lang.type) -> (Box(co.lang.type)) = {}

result := box(polyId); // ✔ works – x is co.lang.type, not a forall type
```

### v2 — Option A: **impredicative:true** in **@co.dap.generic**

Explicit opt-in via the existing annotation. The compiler only permits **forall** instantiation where declared:

```
@co.dap.generic(
 type={T:{variance:invariant, impredicative:true}}
)
box(x T) -> (Box(T)) = {}

polyId co.lang.type = forall(U).(U)->(U)
result := box(polyId); // ✓ legal – impredicative:true explicitly opts in
```

Generic Function Rank Support Matrix

Scenario	Allow?	Notes
Rank-1 generic param (Syntax 1, 2, 3)	✓ Yes	Natural extension, no new concepts
Rank-1 generic return (Syntax 1, 2, 3)	✓ Yes	Same as above
Rank-2 param via forall (Syntax 1, 2)	✓ Yes	co.lang.type naturally holds polymorphic types; no new kind needed
Rank-2 param via Syntax 3 co.lang.function	✗ Compiler error	Function objects are concrete values; use co.lang.type = forall(T).(T)->(T) instead
Rank-2 return via forall (Syntax 1, 2)	✓ Yes	Same reasoning as param
Rank-3 via forall nesting (Syntax 1, 2)	✓ Yes	No new constructs — forall nesting works naturally
Rank-3 return	✓ Yes	Same reasoning as Rank-3 param
Rank-3 via Syntax 3 co.lang.function	✗ Compiler error	Same rule as Rank-2; function objects are concrete
Impredicative — v1 workaround (Option C)	✓ Yes	Wrap forall type in co.lang.type; solves 90% of real cases
Impredicative — true opt-in (Option A)	soon v2	impredicative:true in @co.dap.generic; explicit opt-in

Generic Types

```
@co.dap.generic(typtype=T)
LinkedList co.lang.struct={
 value T
 next LinkedList
 prev LinkedList
}

k := LinkedList.new(co.lang.int);

@co.dap.generic(type={T:{typtype}, R:{typtype}})
Employee co.lang.class = {
 id T
 name R

 @co.dap.override
 @co.dap.constructor(access=private)
 @@init() = {}
}
```

```

 @co.dap.override
 @co.dap.constructor(access=public)
 @@init(id T, name R) = {
 this.parent.@@init();
 this.id = id;
 this.name = name;
 }

 getEmployee(id T)->(Employee)={}
}

a := Employee.new(co.lang.int, co.lang.string);
b := a.@@init(1, "Rao");

```

## Generics Inheritances and Types

This is in conceptual stage not supported.

- A) Abstract vs concrete type members
- B) Path-dependent types
  1. Type-level projection
  2. Path-dependent In folang how it would be

## forall

### What **forall** Is — and Is Not

**forall** is **not** a general-purpose generic declaration keyword. It is a **type-level expression only**, restricted to contexts where a polymorphic type must appear as an anonymous value inline — specifically Rank-2 and Rank-3 parameter and return positions, and `co.lang.type` aliases.

`@co.dap.generic` is the **one and only** way to declare generic functions, structs, classes, and other named things. **forall** at declaration level is a **compiler error**.

---

### Where **forall** Is Allowed — Type Expression Form Only

**forall(T)**. followed by an anonymous type body. The `.` is the syntactic signal that what follows is a type body, not a declaration name.

Pattern:

```
forall(T). <anonymous type body>
```

```

// co.lang.type alias – naming a polymorphic type for reuse
someFArg co.lang.type = forall(T).(T, T)->(T)

// Rank-2 inline parameter – callee decides what T is
someFunction(f forall(T).(T)->(T)) -> (co.lang.int) = {}

// Rank-2 return type – returning a polymorphic function
makeIdentity() -> (forall(T).(T)->(T)) = {}

// Rank-3 inline parameter – f takes a Rank-2 function
applyRank2(f (forall(T).(T, T)->(T)) -> (co.lang.int)) -> (co.lang.int) = {}

```

Where forall Is Banned — Use @co.dap.generic Instead

```
// ✖ compiler error – forall at declaration level
forall(T) identity(x T)->(T) = {}

// ✔ correct
@co.dap.generic(type={T:{variance:invariant}})
identity(x T)->(T) = {}
```

```
// ✖ compiler error
forall(T) LinkedList co.lang.struct = { value T; next LinkedList; }

// ✔ correct
@co.dap.generic(typtype=T)
LinkedList co.lang.struct = { value T; next LinkedList; }
```

```
// ✖ compiler error – Rank-1 generics belong to @co.dap.generic
forall(T) someFunction(f (T,T)->(T), a T)->(T) = {}

// ✔ correct
@co.dap.generic(type={T:{variance:invariant}})
someFunction(f (T,T)->(T), a T)->(T) = {}
```

Quick Reference

Form	Status	Context
forall(T) name ...	✖ Compiler error	Declaration level — use @co.dap.generic instead
forall(T).(T)->(T)	✔ Allowed	Type level only — Rank-2/3 param, return, co.lang.type alias

The rule in one sentence: forall(T). is a type constructor for anonymous polymorphic types; it is never a declaration keyword.

Templates

Typed

```
@co.dap.template
add(a co.lang.int, b co.lang.int)->(co.lang.int) ={
 this.return a + b;
}
```

Untyped

```
@co.dap.template
add(a, b)->(co.lang.untyped) ={
 this.return a + b;
}
```

## Macros

```
// a. Basic macro
@co.dap.macro
say()->()={ this.return co.macro.quote({ println("Line 1") println("Line 2") }); }

// b. Escape assign
@co.dap.macro
yes_esc_assign()->(co.lang.untyped)={
 this.return co.macro.quote({
 co.macro.esc(y) = 42
 println("Inside macro: y = ", y)
 });
}

// c. Debug macro with gensym
@co.dap.macro
debug(expr)->(co.lang.untyped)={
 let tmp = co.macro.gensym(co.lang.var, "tmp")
 this.return co.macro.quote({
 tmp = co.macro.esc(expr)
 println("Result: ", tmp)
 tmp
 });
}

// d. if/else condition macro
@co.dap.macro(
 group = {items:["if","else"], chain:true},
 sugarform={forms:["if expr block"]},
 bind={vars:["x"]},
 isolate={vars:["temp", "index"]},
 gensym={prefix:"tmp_"},
 hygienic=true,
 argtransform={param:"body", wrap:"lambda", whentype:"block"},
 desugar={exprs:["if($cond) { $block }" => "if($cond,$block)"]},
 mode="inject"
)
if(condition expr, body block)->()={}

blockormacro co.lang.Kind = block | macro

@co.dap.macro(
 group={items:["if","else"], chain:true},
 sugarform={forms:["else block","else if"]},
 chainswith={macro:"if", position:"immediate", required:true},
 argtransform={param:"body", wrap:"lambda", whentype:"block"},
 standalone=false,
 desugar={exprs:[
 "else if($cond) { $block }" => "else(if($cond, $block))",
 "else { $elseblock }" => "else($elseblock)"
]},
)
else(body blockormacro)->()={}

```

Other macro utilities:

1. `@co.dap.compose(using=["base_if", "blockify"])`

2. `@co.dap.guard(expr="is_bool_expr(expr)")`
3. Quasiquote macros use `co.macro.quote` and `co.macro.unquote`

## Annotations, Directives, Pragmas and Decorators

```
// Annotation – static object, can carry data

myAnnotation co.lang.object->(for=annotation) = {
 value co.lang.string;
 enabled co.lang.bool;
}

// Directive – static object, compiler instruction, can carry data
myDirective co.lang.object->(for=directive) = {
 target co.lang.string;
}

// Pragma – static object, compiler hint, can carry data
myPragma co.lang.object->(for=pragma) = {
 level co.lang.int;
}

// Decorator – function, transforms target, returns
@co.dap.decorator
myDecorator(target co.lang.function)->(co.lang.function) = { }
```

## Pattern Matching

```
x co.lang.int = 10;

x.match.case(n: n > 10 => { n = n+100; "GT" }).case(_: n < 10 => "LT").default("EQ");

x.match(co.pattern.Type).case(co.lang.int => ...).case(co.lang.float => ...);
x.match(co.pattern.Value).case(0 => ...).case(1 => ...);
x.match(co.pattern.Instance).case(xx.CAT => ...).case(xx.DOG => ...).default("Animal");
x.match(co.pattern.Object).case(xx.Ball => "Ball").case(xx.CAT =>
"CAT").default("Unknown");
x.match(co.pattern.Shape).case(Point{x, y} => ...).default(_ => ...);

x.match(co.pattern.Any).case(co.lang.int => ...).case(co.lang.float => ...).case(0 =>
...).default(_ => ...);

x.match(PositiveEvenMatcher).case(0 => "Neither even nor odd").case(2 => "First Even
Prime").default(...);
```

**Object vs Instance in FoLang:** Instance is from types of class/structs. Objects are anything — functions, classes, structs, types, etc.

`_` is a special discard/wildcard variable usable only inside pattern matching, contains, and iterator constructs. Elsewhere `_` must be accompanied by an ASCII letter or number.

### Custom Matcher

```
@co.dap.matcher
Matcher(T) = {
 matchCase(value T, pattern co.lang.untyped) -> (co.lang.int, co.lang.MatchBindings);
 // int return: 0 = no match, >0 = match
}

PositiveEvenMatcher co.lang.Matcher->(for=Matcher, type=co.lang.int) = {
 matchCase(value co.lang.int, pat co.lang.untyped)->(co.lang.int,
co.lang.MatchBindings) = {
 // user logic
 }
}
```

## Function Pattern

```
f(Some(x)) => { x + 1 }
f(None()) => { 0 }

// desugars to:
f(v) =>{
 v.match().case(x: Some(x) => x + 1).case(_: None() => 0);
}
```

---

## Monads, Applicatives, Functors, Monoids and Transformers

`@co.dap.typeclass(kind=...)` is the single annotation for all typeclass definitions. `kind` specifies the algebraic structure — `Functor`, `Applicative`, `Monad`, `Monoid`, `Transformer`, or any user-defined kind. Instances of any typeclass always use `co.lang.instance`.

### Functor

```
@co.dap.typeclass(kind=Functor)
Functor(F) = {
 map(value F(A), f (A)->B) -> (F(B));
}

ListFunctor co.lang.instance->(for=Functor, type=List) = {
 map(value List(A), f (A)->B)->(List(B)) = {
 result = List(B){}
 value.each(_, item).do({ result.append(f(item)) });
 this.return result
 }
}
```

### Applicative

```
@co.dap.typeclass(kind=Applicative)
Applicative(F) = {
 pure(x A) -> (F(A));
 apply(fab F(A->B), fa F(A)) -> (F(B));
}

OptionApplicative co.lang.instance->(for=Applicative, type=Option) = {
 pure(x A)->(Option(A)) = { this.return Some(x); }
```

```

 apply(fab Option(A->B), fa Option(A))->(Option(B)) = {
 this.return (fab, fa)
 .match
 .case((Some(f), Some(x)) => Some(f(x)))
 .default(None());
 }
 }
}

```

## Monad

```

@co.dap.typeclass(kind=Monad)
Monad(F) = {
 pure(x A) -> (F(A));
 flatMap(fa F(A), f (A)->F(B)) -> (F(B));
}

OptionMonad co.lang.instance->(for=Monad, type=Option) = {
 pure(x A)->(Option(A)) = { this.return Some(x); }
 flatMap(fa Option(A), f (A)->Option(B))->(Option(B)) = {
 this.return fa.match().case(Some(x) => f(x)).default(None);
 }
}

```

## Monoid

```

@co.dap.typeclass(kind=Monoid)
Monoid(T) = {
 empty() -> (T);
 combine(a T, b T) -> (T);
}

IntMonoid co.lang.instance->(for=Monoid, type=co.lang.int) = {
 empty()->(co.lang.int) = { this.return 0; }
 combine(a co.lang.int, b co.lang.int)->(co.lang.int) = { this.return a + b; }
}

```

## Transformer

```

@co.dap.typeclass(kind=Transformer)
Transformer(F(_), G(_)) = {
 map(value F(A), f (A)->B) -> (G(B));
}

ListToSetTransformer co.lang.instance->(for=Transformer, types=[List, Set]) = {
 map(value List(A), f (A)->B)->(Set(B)) = {
 result = Set(B){}
 value.each(_, item).do({ result.insert(f(item)) });
 this.return result;
 }
}

```

## Let Bindings

```

y co.lang.int = let({x = 10}).in({x + 1});
y co.lang.int = let({$ = 10}).in({$ + 1}); // $ refers to the value being defined

x co.lang.int = (x + 1).where(x = 10);
x co.lang.int = ($ + 1).where($ = 10);

let fib(0) = 1
let fib(1) = 1
let fib(n) = fib(n-1) + fib(n-2)

```

`$` is a special identifier usable in `let` bindings for recursive or self-referential expressions.

## Conditions, Loops and Iterators

### Conditions

```

(boolean truth).do({
}).otherwise(boolean truth).do({
}).otherwise.do({
});

```

### Loops

```

(boolean truth).loop({
}).otherwise(boolean truth).loop({
}).otherwise.loop({
});

```

### Condition and Loop Mix

```

(boolean truth).do({
}).otherwise(boolean truth).loop({
}).otherwise(boolean truth).do({
}).otherwise.loop({
});

```

### Ternary Operator

```

s = (boolean truth).return(some var/value).otherwise.return(some val/var);
s = (boolean truth).return(some var/val).otherwise(boolean truth).return(some
var/val).otherwise.return(some var/val);

```

### Looping Arrays / Lists / Maps / Ranges

```

arr co.lang.int->([5]) = [6,7,8,9,10];
arr.each(idx, val).do({
 co.out.print(idx);
 co.out.print(" :: ");
 co.out.println(val);
});

```

```
arr.each(_, val).do({
 co.out.println(val);
});
```

## Array / List / Map / Range — Contains Element

```
arr co.lang.int->([5]) = [35,57,96,81,31];
k co.lang.int = 31;
arr.contains(k).do({
 co.out.println(k);
}).otherwise.do({
 co.out.println("Not Found");
});
```

## Comprehensions (*planned*)

```
k := (1..10).filter(|x| => x % 2 == 0).map(|x| => x * x);

result := for (x <- List(1,2,3)).yield(x * 2) // List(2, 4, 6)
result := for (x <- Set(1,2,3)).yield(x * 2) // Set(2, 4, 6)
result := for (x <- Some(5)).yield(x * 2) // Some(10)
result := for (x <- fetchData()).yield(x.process()) // Future

ages := {"A":30,"B":40,"C":66,"E":88};
upper := for ((name, age) <- ages).yield(name.toUpperCase, age)
```

---

## Extensions

```
@co.dap.extension(fortype=co.lang.string, what=extends)
upperCase()->(string)={
 return this.upper()
}

@co.dap.extension(fortype=[co.lang.string], what=overrides)
equals(str string)->(bool)={
 this.return this == str
}
```

Extensions must be **explicitly activated** — they are block-scoped:

```
@co.ddap.use(extensions=[equals, upperCase])
k.upperCase(); // ✓ explicitly activated
```

---

## Labels and Named Blocks

```
// Label
outer:{
 // statements
}
```

```
// Named Block
labelBlock co.lang.block={

}

labelBlock.inline();
```

---

## Reflection

```
@co.dap.reflection(enable=True, package="co.meta")

x co.lang.int = 10;
x.reflect().getType() → co.lang.int
x.reflect().getValue() → 10
x.reflect().getKind() → value
```

---

## Forward / Extern Declarations

### Variable

```
@co.dap.extern
someBool co.lang.bool;
```

### Functions

```
@co.dap.extern
getEmployee(id co.lang.int)->(somepack.Employee);

// or – @co.dap.extern is optional for functions
getEmployee(id co.lang.int)->(somepack.Employee);
```

### Types

```
@co.dap.extern
Employee co.lang.struct;

// or – @co.dap.extern is optional for types
Employee co.lang.struct;
```

For functions and types `@co.dap.extern` is optional. For variables it is required.

---

## Interface vs Signature

```
MEmployee co.lang.signature = {
 Employee co.lang.struct;
 storeEmployee(emp Employee) -> (Employee);
}
```

```
IEmployee co.lang.interface = {
 storeEmployee(emp Employee) -> (Employee);
}
```

Structurally they look similar — both are lists of contracts. The difference is **who implements them and how**.

	co.lang.signature	co.lang.interface
Implemented by	module via <b>matches=</b>	class via <b>implements=[]</b>
Can include types/structs	✓	×
Instantiation involved	×	✓
OOP dispatch	×	✓ virtual/dynamic
Behavior only	×	✓ (like Go)
Origin	ML/OCaml modules	Java/C#/Go interfaces

- A **signature** is a **structural contract** — can include types, structs, nested definitions. Describes a whole capability unit.
- An **interface** is a **behavioral contract** — methods only, no fields, no type declarations. Tied to OOP dispatch and polymorphism.

## Structs vs Classes vs Modules vs Packages

	Struct	CStruct	Class	Module	Package
Purpose	Pure data shape	C-like value type	Behavior + data	Named function bundle	Namespace grouping
Fields	✓	✓ simple only	✓ per instance	×	×
Methods	×	×	✓	✓ free functions	×
Lifecycle ( <b>@@new/@@init</b> )	×	×	✓	×	×
<b>this / self</b>	×	×	✓	×	×
Instantiable	×	×	✓ multiple objects	×	×
First class	✓	✓	✓	✓	×
Pass by	Reference	Value	Reference	Reference	—
Implements	×	×	<b>interface</b> via <b>implements=[]</b>	<b>signature</b> via <b>matches=</b>	×
OOP / inheritance	×	×	✓	×	×
Contains type declarations	✓ struct/enum	×	✓ inner, <b>Class.Type</b>	✓ via signature	✓ any
Pattern matching	✓	✓	✓	×	×
Zone boundary safe	×	✓	×	×	×
Associated functions	✓	×	—	—	×

	Struct	CStruct	Class	Module	Package
Embedding	✓	×	—	—	×
Declared with	co.lang.struct	co.lang.cstruct	co.lang.class	co.lang.module	folder path is package by default
Contract type	—	—	co.lang.interface	co.lang.signature	—
C++ backend	struct (no methods)	plain C struct	class	struct/class	namespace
Closest analogy	Rust struct	C struct	Java / C# class	OCaml module	C++ namespace

Mental model:

reach for struct

→ pure data, no behaviour

reach for cstruct

→ data crossing zone boundaries

reach for class

→ behaviour, lifecycle, multiple instances

reach for module

→ named function bundle, single implementation

reach for package

→ grouping only, compile time, not a value

**Type declaration scoping rule:** Modules own types as part of their public contract via signature. Classes own inner types accessible via `ClassName.TypeName`. Functions may declare types locally — local types are scoped to the function body only and cannot appear in parameter or return types. Structs cannot declare types — they are pure data with no scope.

## Built-in Types

Type	Kind
co.lang.string	
co.lang.int	
co.lang.bit	
co.lang.double	
co.lang.float	
co.lang.long	
co.lang.byte	
co.lang.char	
co.lang.any	
co.lang.dynamic	
co.lang.auto	
co.lang.infer	
co.lang.bool	
co.lang.void	
co.lang.data	

Type	Kind
co.lang.value	
co.lang.typed	
co.lang.untyped	
co.mem.region	
co.lang.nothing	
co.lang.word	
co.lang.MatchBindings	
co.lang.tag	
co.lang.typevalue	
co.lang.uninit	
co.lang.Literal	

## Built-in kinds

Kind	Purpose
co.lang.type	
co.lang.struct	
co.lang.cstruct	
co.lang.realm	similar to loader where symbols reside
co.lang.loader	class, type, functions loader where objects reside loader takes realm as parameter
co.lang.class	
co.lang.interface	
co.lang.union	
co.lang.role	
co.lang.record	
co.lang.property	
co.lang.indexer	
co.lang.object	
co.lang.instance	
co.lang.matcher	
co.lang.trait	
co.lang.mixin	
co.lang.extension	
co.lang.delegate	
co.lang.typeclass	
co.lang.concept	

Kind	Purpose
co.lang.typealias	
co.lang.module	
co.lang.macro	
co.lang.template	
co.lang.lambda	
co.lang.block	
co.lang.behavior	
co.lang.package	
co.lang.signature	
co.lang.function	
co.lang.method	
co.lang.operator	
co.lang.namespace	
co.lang.stex	
co.lang.kind	
co.lang.level	
co.lang.order	
co.lang.rank	
co.lang.newtype	
co.lang.opaquetype	
co.lang.subtype	
co.lang.supertype	
co.lang.dependentType	
co.lang.refinementType	
co.lang.associatedtype	
co.lang.hkt	higher kind type
co.lang.hot	higher orderr type
co.lang.hrt	higher rank type
co.lang.data	
co.lang.enum	
co.lang.typetype	
co.lang.typekind	
co.lang.alias	
co.lang.value	
co.lang.just	

Kind	Purpose
co.lang.nothing	

## Built-in Directives

Kind	
PRAGMA	"@co.pdap.compiler", "@co.pdap.scale"
DIRECTIVE	"@co.ddap.movetotop", "@co.ddap.import", "@co.ddap.dynamicruntime", "@co.ddap.use", "@co.ddap.parent", "@co.ddap.alias"
ANNOTATION	"@co.dap.template", "@co.dap.macro", "@co.dap.operator", "@co.dap.annotation", "@co.dap.library", "@co.dap.module", "@co.dap.pragma", "@co.dap.directive", "@co.dap.native", "@co.dap.classmethod", "@co.dap.static", "@co.dapinstancemethod", "@co.dap.objectmethod", "@co.dap.inline", "@co.dap.ctfe", "@co.dap.friend", "@co.dap.sealed", "@co.dap.extension", "@co.dap.override", "@co.dap.virtual", "@co.dap.abstract", "@co.dap.delegate", "@co.dap.dynamicscope", "@co.dap.lexicalscope", "@co.dap.staticscope""@co.dap.mixedscope", "@co.dap.typeclass", "@co.dap.matcher", "@co.dap.constructor", "@co.dap.oops", "@co.dap.hokrt", "@co.dap.hokrtl", "@co.dap.indexer", "@co.dap.generic", "@co.dap.comptime", "@co.dap.typefromvalue", "@co.dap.export", "@co.dap.eager", "@co.dap.lazy", "@co.dap.packed", "@co.dap.declare", "@co.dap.simd", "@co.dap.reflection", "@co.dap.mop"//mop => meta object programming
DECORATOR	"@co.dap.before", "@co.dap.after", "@co.dap.around", "@co.fx.onErrExcept", "@co.fx.InvokeAlways", "@co.fx.HandleEffect", "@co.dap.callback", "@co.dap.defer", "@co.dap.continuation", "@co.dap.event", "@co.dap.scale", "@co.dap.distributed", "@co.dap.concurrent", "@co.dap.parallel", "@co.dap.subroutine", "@co.dap.generator", "@co.dap.goroutine", "@co.dap.coroutine", "@co.dap.async", "@co.dap.promise", "@co.dap.future", "@co.dap.thread", "@co.dap.task", "@co.dap.fiber", "@co.dap.process", "@co.dap.spawn", "@co.dap.exec", "@co.dap.fork", "@co.dap.csp", "@co.dap.actor", "@co.dap.synthetic", "@co.dap.bridge", "@co.dap.greenlet", "@co.dap.channel", "@co.dap.callable", "@co.dap.iterator"

## Built-in Packages

co — root (reserved word)

The only package provided by default.

Sub-package	Responsibility
co.lang	All data types and kinds
co.sys	file, concurrent, parallel, goto, invoke, bind, call, apply, setTimeout, setInterval, scheduler, cron, event
co.os	signal, cmd, execute, run, env, getenv, setenv, sleep, exit, cwd, chdir, fork, wait, pipe, dup, close, readfd, writefd
co.meta	"ast", "instrument", "transform", "augment", "reflect", "introspect", "patch", "inject", "create", "runtime(eval,etc)", "realm"
co.core	list, set, map, tree, tries, sort, search, matrix
co.native	load, register, asm, inline, emit, ffi,spawnnon[gpu,cpu,mpu,apu,fpga,asic,tpu,mki,mcu],arch[x86,x86-64,risc,arm,vliw]

Sub-package	Responsibility
co.in	read, readln
co.out	println, print
co.regex	stex, pattern, match, search
co.crypto	rsa, aes, hash, md5, rand, uuid, ssl, tls
co.dap	built-in decorators, annotations
co.ddap	built-in directives
co.pdap	built-in pragmas
co.net	tcp, udp, http
co.const	true, false, none
co.encoding	base64Encode, base64Decode, json, yml, bson
co.utils	makeImmutable, makeShared, copyOnWrite, toLiteral — object behaviour policies
co.dynamic	dynamic capabilities
co.runtime	
co.compiletime	
co.macro	
co.pattern	
co.control	
co.cpa	concurrent, async, await, defer, lazy, parallel, process, thread, fiber, task, coroutine, continuation, cps, etc.
co.hokrtl	
co.hokrt	